



Estructura de Datos.
Examen Convocatoria Ordinaria
Duración: 2:30 horas

DNI:

Nombre:

TEORÍA (5 PUNTOS)

1.- (2,5 puntos) Responde a las siguientes preguntas relacionada con la eficiencia de algoritmos:

- a. **(1 puntos)** Enumera y explica suficientemente que notaciones existen para la complejidad asintótica.
- b. **(1,5 puntos)** Dado el siguiente algoritmo:

```
void Insercion(int numbers[], int array_size) {  
    int i, a, index;  
    for (i=1; i < array_size; i++) {  
        index = numbers[i];  
        a = i-1;  
        while (a >= 0 && numbers[a] > index) {  
            numbers[a + 1] = numbers[a];  
            a--;  
        }  
        numbers[a+1] = index;  
    }  
}
```

- a) **(1 puntos)** ¿Que eficiencia tiene el método? ¿Por qué?
- b) **(0,5 puntos)** Si el algoritmo anterior lo programo en Java, ¿Será más rápido que el proporcionado en C++? ¿Será más eficiente? Explica y justifica suficientemente tu respuesta.

2.- (2,5 puntos) Resuelva las siguientes cuestiones:

- a) **(1 punto)** Partiendo de un árbol APOC de números enteros vacío representar, dibujando la evolución del árbol, cómo se realizan las siguientes acciones (en el orden en el que se listan).

- Insertar los enteros 3, 9, 2, 5, 1, 7, 4, 6, 11, -4.
- Listar los nodos siguiendo los recorridos en preorden, inorden y postorden.

- b) **(1,5 puntos)** Partiendo de un árbol AVL de números enteros vacío representar, dibujando la evolución del árbol, cómo se realizan las siguientes acciones (en el orden en el que se listan).

- Insertar los enteros 10, 100, 20, 80, 40, 70.
- Borrar 100, Borrar 40.
- Insertar 35, insertar 60.
- Listar los nodos siguiendo los recorridos en preorden, inorden y postorden.

PRÁCTICAS (5 PUNTOS)

3.- (2,5 puntos) Dado un árbol general con interfaz:

```
template <class T>
class Arbol {
public:
    Arbol();
    Arbol(T valor);
    Arbol(const Arbol<T> & a);
    ~Arbol();
    void setRaiz(T valor);
    Nodo getRaiz();
    T getEtiqueta(Nodo n);
    void setEtiqueta(Nodo &n, T valor);
    Nodo getPadre(Nodo n);
    Nodo getHijoIzq(Nodo n);
    Nodo getHermanoDcha(Nodo n);
    Nodo getHermanoIzq(Nodo n);
    void insertarHijoIzq(Nodo &p, T valor);
    void insertarHermanoDcha(Nodo &p, T valor);
    void vaciar();
    bool estaVacio() const;
    Arbol<T> & operator= (const Arbol<T> &a);
    bool operator== (const Arbol<T> &a) const;
};
```

- (1,5 punto) Sobrecargue e implemente el operador de salida << para que muestre por pantalla el recorrido en preorden del árbol.
- (0,5 puntos) Escribir una función, que pasados un árbol y uno de sus nodos como argumentos, calcule la altura del nodo en el árbol.
- (0,5 puntos) Escribir otra función para calcular la profundidad del nodo en el árbol.

4.- (2,5 puntos)

- **(1,5 punto)** Se desea implementar el TDA Cola utilizando el TDA Pila. Se debe utilizar el tipo abstracto de datos de la STL que se corresponda con una pila (queue). Especifica la interfaz del TDA, función de abstracción e invariante de representación en el cual se indique la especificación del mismo y todas las funciones necesarias para que funcione correctamente.
- **(1 punto)** Implementa la función de inserción y borrado de la Cola. ¿Qué eficiencia tienen dichos métodos?